

Milestone Four Narrative

Sabrina Ozburn

CS-499 Computer Science Capstone

Prof. Wasim Alim

8/1/2026

Briefly describe the artifact. What is it? When was it created?

The artifact selected for enhancement is a full-stack, data-driven web application dashboard developed for Grazioso Salvare, an international animal rescue organization. Created in 2025 during the CS 340 Client-Server Development course, this application uses Python Dash and Plotly to render interactive filtering tables, demographic pie charts, and real-time geospatial coordinate-tracking maps. The system allows rescue personnel to dynamically filter for specialized animal populations, such as water-rescue or disaster-tracking dogs, providing crucial data visualizations to support critical organizational decision-making.

Justify the inclusion of the artifact in your ePortfolio. Why did you select this item? What specific components of the artifact showcase your skills and abilities in software development? How was the artifact improved?

This artifact is included in my ePortfolio because it demonstrates my technical ability to enforce backend data governance and implement advanced data-layer query-optimization models, which are vital skills for a professional backend engineer or database administrator. To showcase a rigorous approach to system administration and technical data integrity, the artifact was improved in two specific areas:

- **Server-Side JSON/BSON Schema Validation Routine:** I engineered a proactive internal gatekeeper function inside the create method of the AnimalShelter module. This routing schema tests all input arguments for structural validity and completeness before executing any write transactions. It verifies mandatory fields and restricts input parameters to strict data types, ensuring that empty strings or malformed records are instantly blocked at the application threshold.

- **Compound Query Performance Optimization Keying:** I built a specialized, multi-key composite index architecture layout (`{'breed': 1, 'age_upon_outcome_in_weeks': 1}`). This structure simulates how a physical database engine optimizes query processing speed by transforming resource-heavy sequential table sweeps into a fast, logarithmic $O(\log n)$ lookup transaction that complements the upstream QuickSort sorting loops.

Did you meet the course outcomes you planned to meet with this enhancement in Module One? Do you have any updates to your outcome-coverage plans?

This technical database enhancement successfully met and advanced the course outcomes originally targeted in Module One:

- **Outcome 3: Design and evaluate computing solutions... managing design trade-offs.**
This was achieved by configuring an optimized compound indexing scheme that prioritizes rapid read operations and local array slicing, directly managing the structural resource trade-off between database server computing overhead and presentation-layer responsiveness.
- **Outcome 5: Develop a security mindset... to mitigate design flaws and ensure security of data.** This was achieved by deploying strict server-side document verification rules that anticipate potential security gaps or adversarial exploits. By actively intercepting malformed payloads or invalid inputs before they reach core data storage collections, the system eliminates vulnerabilities such as input injection and data corruption.

Reflect on the process of enhancing and modifying the artifact. What did you learn as you were creating it and improving it? What challenges did you face?

The process of modifying and expanding this database artifact provided valuable insight into professional data stewardship and security design workflows. I learned that robust software relies heavily on proactive backend data protection layers rather than assuming client inputs will always arrive flawlessly. A major technical challenge faced during development was integrating a data governance validation routine that remained fully compatible with a native, local execution script layout without introducing severe performance bottlenecks or crashing interface graph metrics. Overcoming this roadblock involved designing clean, conditional validation logic checks that evaluate dictionary key parameters seamlessly during local testing cycles. This challenge taught me how to trace data mutations across the data access tier and enforce strict automated testing standards, which has significantly boosted my confidence as I prepare for professional roles in database administration and software quality assurance.